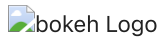


Bokeh for Time Series Analysis



****Introduction to Bokeh**** Bokeh is an interactive visualization library for Python that targets modern web browsers for presentation. Unlike Matplotlib, which is primarily designed for static plots, Bokeh excels at creating interactive plots and dashboards. It can handle large datasets and streaming data, making it suitable for real-time applications.

Key Features of Bokeh:

- **Interactivity:** Built-in support for zooming, panning, hovering, and other interactive tools.
- **Web-Focused:** Generates HTML and JavaScript, making it easy to embed plots in web pages.
- **High Performance:** Can handle large datasets efficiently.
- **Versatility:** Supports a wide range of plot types (lines, bars, scatter plots, etc.).

Documentation:

For comprehensive documentation, please refer to the official Bokeh website: <https://docs.bokeh.org/en/latest/>

Lab Exercise:

Your task is to recreate the time series analysis lab we previously conducted using Pandas, Matplotlib, and Seaborn, but this time, utilize the Bokeh library for visualization. This will involve:

1. Loading and preprocessing the "AirPassengersDates.csv" dataset.
2. Creating interactive Bokeh plots for:
 - Time series line plots
 - Bar plots of aggregated data
 - Visualizing mean and standard deviation
 - Outlier detection
 - Resampling (upsampling and downsampling)
 - Lag analysis
 - Autocorrelation

Pay close attention to Bokeh's features for interactivity (tools, hover effects) and its handling of data sources. Aim to replicate the insights and visualizations from the previous lab while leveraging Bokeh's strengths.

Good luck!

```
In [9]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import bokeh.plotting as bp
from bokeh.io import output_notebook
import numpy as np

output_notebook() # Affiche les plots inline dans Jupyter

passenger_df = pd.read_csv('../dataset/passengers-date.csv')
passenger_df.head()
```



BokehJS 3.8.2 successfully loaded.

Out[9]:

	Date	#Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

Time series line plots

```
In [10]: passenger_df["Date"] = pd.to_datetime(passenger_df["Date"])

print(passenger_df.info())

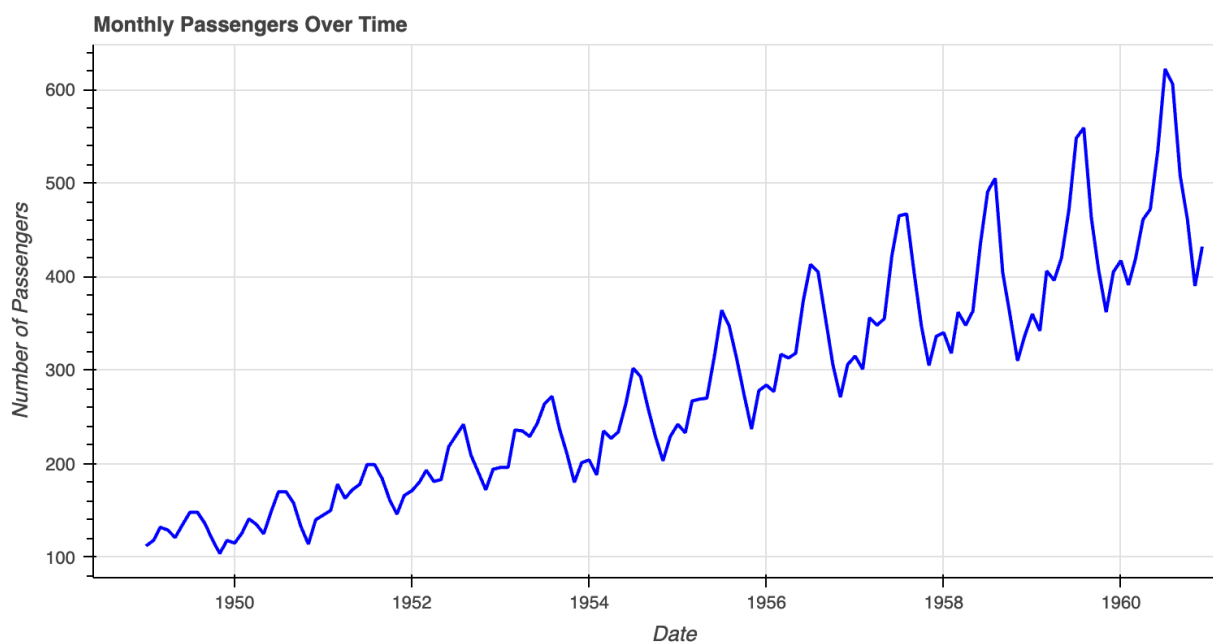
passenger_df["Month"] = passenger_df["Date"].dt.month
passenger_df["Day"] = passenger_df["Date"].dt.day
passenger_df["Day_Name"] = passenger_df["Date"].dt.day_name()

print(passenger_df.head())

p = bp.figure(title="Monthly Passengers Over Time", x_axis_type='datetime', width=800, height=
p.line(passenger_df["Date"], passenger_df["#Passengers"], line_width=2, color='blue')
p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Number of Passengers"
bp.show(p)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Date            144 non-null   datetime64[ns]
1   #Passengers      144 non-null   int64
dtypes: datetime64[ns](1), int64(1)
memory usage: 2.4 KB
None
```

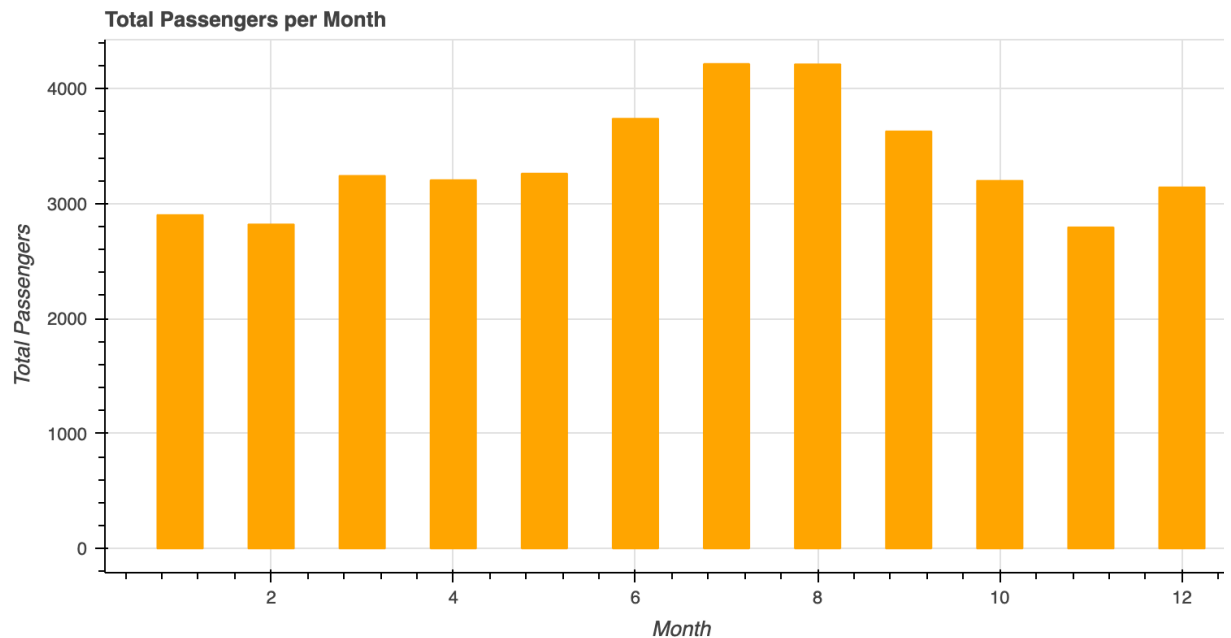
```
   Date      #Passengers  Month  Day  Day_Name
0 1949-01-01         112      1    1  Saturday
1 1949-02-01         118      2    1  Tuesday
2 1949-03-01         132      3    1  Tuesday
3 1949-04-01         129      4    1  Friday
4 1949-05-01         121      5    1  Sunday
```



Bar plots of aggregated data

```
In [11]: passengers_per_month = (
    passenger_df.groupby("Month")["#Passengers"].sum().reset_index()
)

p = bp.figure(title="Total Passengers per Month", x_axis_label='Month', y_axis_label='Total Pa
p.vbar(x=passengers_per_month["Month"], top=passengers_per_month["#Passengers"], width=0.5, co
bp.show(p)
```



Visualizing mean and standard deviation

```
In [12]: monthly_stats = passenger_df.groupby("Month")["#Passengers"].agg(['mean', 'std']).reset_index()
monthly_stats['upper'] = monthly_stats['mean'] + monthly_stats['std']
monthly_stats['lower'] = monthly_stats['mean'] - monthly_stats['std']

p = bp.figure(
    title="Moyenne et Écart-Type des Passagers par Mois",
    x_axis_label='Mois',
    y_axis_label='Nombre de Passagers',
    width=800,
    height=400
)

p.varea(
    x=monthly_stats['Month'],
    y1=monthly_stats['lower'],
    y2=monthly_stats['upper'],
    fill_alpha=0.3,
    fill_color='blue',
    legend_label='Écart-type (± 1 std)'
)

p.line(
    x=monthly_stats['Month'],
    y=monthly_stats['mean'],
    color='red',
    line_width=2,
    legend_label='Moyenne'
)

p.scatter(
    x=monthly_stats['Month'],
    y=monthly_stats['mean'],
    color='red',
```

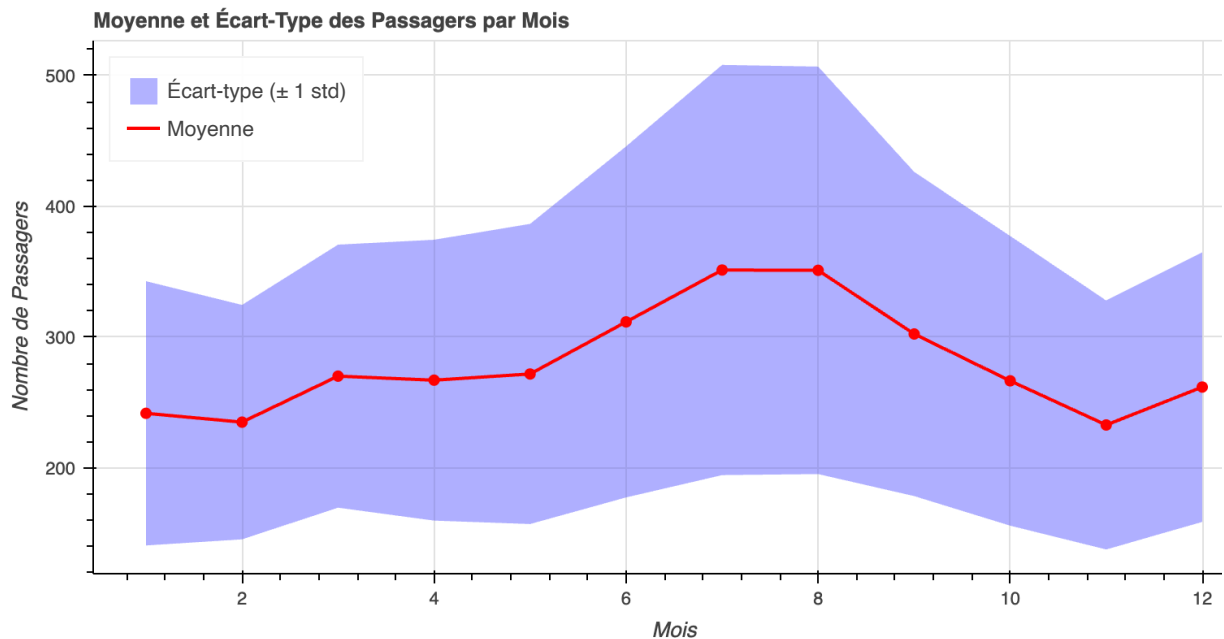
```

size=6
)

p.legend.location = "top_left"

bp.show(p)

```



Outlier Detection (Détection des valeurs aberrantes)

```

In [13]: mean_passengers = passenger_df["#Passengers"].mean()
std_passengers = passenger_df["#Passengers"].std()

threshold = 2
lower_bound = mean_passengers - threshold * std_passengers
upper_bound = mean_passengers + threshold * std_passengers

outliers = passenger_df[(passenger_df["#Passengers"] < lower_bound) | (passenger_df["#Passengers"] > upper_bound)]

p = bp.figure(
    title="Détection des valeurs aberrantes (Outliers)",
    x_axis_type='datetime',
    width=800,
    height=400
)

p.line(
    passenger_df["Date"],
    passenger_df["#Passengers"],
    color='blue',
    line_width=2,
    legend_label='Passagers'
)

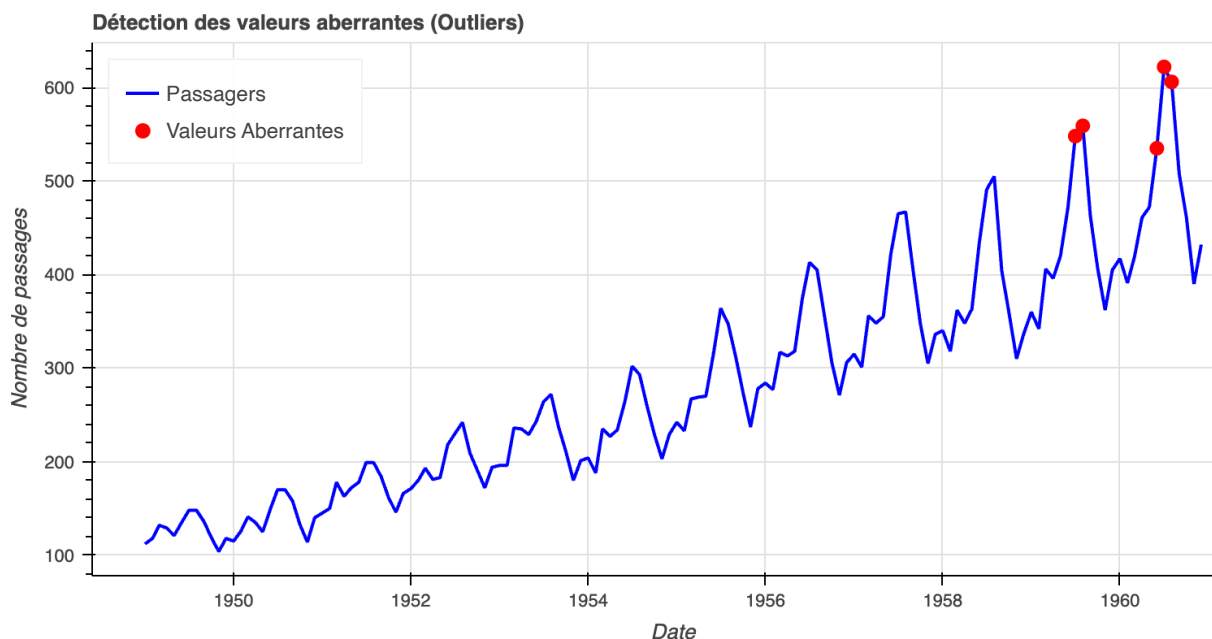
p.circle(
    outliers["Date"],
    outliers["#Passengers"],
    color='red',
    size=8,
    legend_label='Valeurs Aberrantes'
)

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Nombre de passages"
p.legend.location = "top_left"

bp.show(p)

```

BokehDeprecationWarning: 'circle()' method with size value' was deprecated in Bokeh 3.4.0 and will be removed, use 'scatter(size=...) instead' instead.



Resampling (Upsampling et Downsampling)

```
In [14]: df_time = passenger_df.set_index("Date")

downsampled = df_time["#Passengers"].resample('YE').mean()

upsampled = df_time["#Passengers"].resample('D').interpolate(method='linear')

p = bp.figure(
    title="Resampling : Données Originales, Upsampling (Jour) et Downsampling (Année)",
    x_axis_type='datetime',
    width=800,
    height=400
)

p.line(
    upsampled.index,
    upsampled.values,
    color='lightgray',
    line_dash='dotted',
    legend_label='Upsampled (Jour)'
)

p.line(
    df_time.index,
    df_time["#Passengers"],
    color='blue',
    line_width=2,
    alpha=0.6,
    legend_label='Original (Mois)'
)

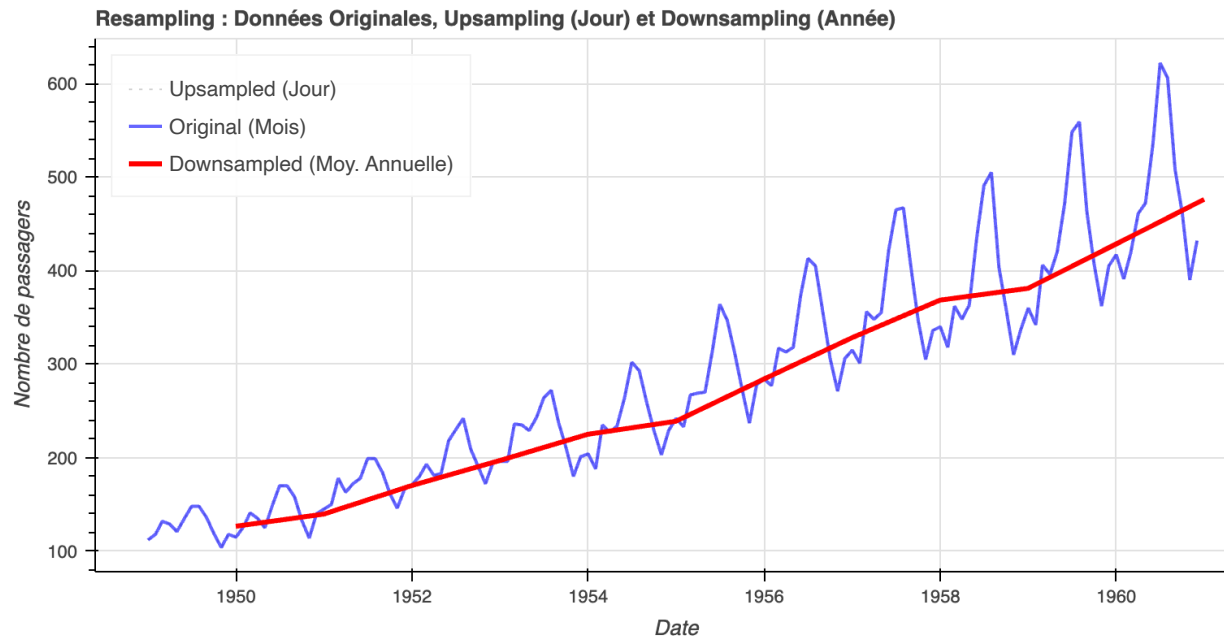
p.line(
    downsampled.index,
    downsampled.values,
    color='red',
    line_width=3,
    legend_label='Downsampled (Moy. Annuelle)'
)
```

```

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Nombre de passagers"
p.legend.location = "top_left"

bp.show(p)

```



Lag Analysis

```

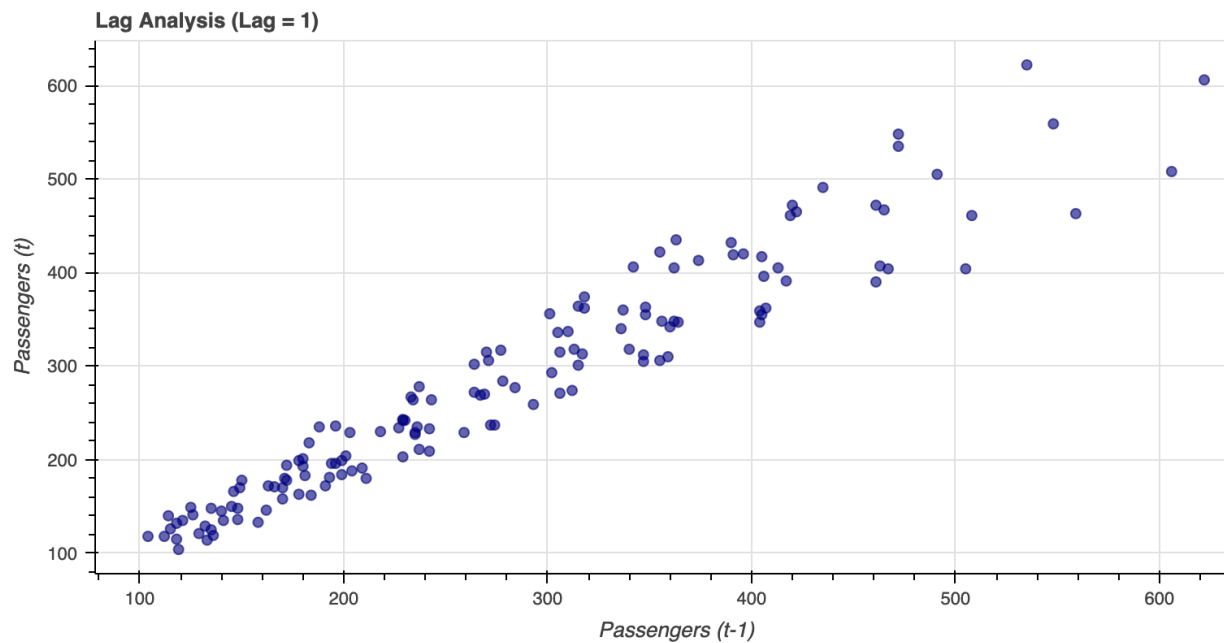
In [15]: lag_1 = passenger_df["#Passengers"].shift(1)

p = bp.figure(
    title="Lag Analysis (Lag = 1)",
    x_axis_label="Passengers (t-1)",
    y_axis_label="Passengers (t)",
    width=800,
    height=400
)

p.scatter(
    x=lag_1,
    y=passenger_df["#Passengers"],
    color='navy',
    size=6,
    alpha=0.6
)

bp.show(p)

```



Autocorrelation

```
In [16]: nlags = 40

acf_values = [passenger_df["#Passengers"].autocorr(lag=k) for k in range(nlags + 1)]
acf_values[0] = 1.0
lags = list(range(nlags + 1))

n = len(passenger_df)
conf = 1.96 / np.sqrt(n)

p = bp.figure(
    title="Autocorrélation (ACF) - Passagers Mensuels",
    x_axis_label="Lag",
    y_axis_label="Autocorrélation",
    width=800,
    height=400
)

p.line([0, nlags], [0, 0], color="black", line_width=1)

p.line([0, nlags], [conf, conf], line_dash="dashed", color="red", line_width=1.5, legend_label="Conf")
p.line([0, nlags], [-conf, -conf], line_dash="dashed", color="red", line_width=1.5)

p.segment(
    x0=lags, y0=0,
    x1=lags, y1=acf_values,
    line_width=2,
    color="navy"
)

p.scatter(
    x=lags,
    y=acf_values,
    size=8,
    color="navy",
    legend_label="ACF"
)

p.legend.location = "top_right"
bp.show(p)
```

